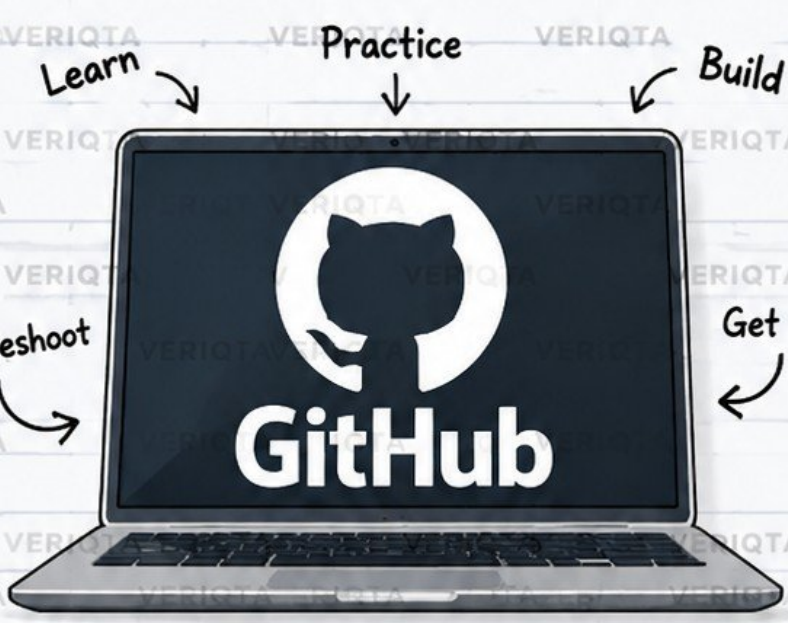
VERIQTA

10 GitHub Repos That Will Get You Hired in DevOps.



Linux

Docker

GitHub Actions

Terraform

AWS

Kubernetes

Prometheus

Argo CD

Helm

Grafana

Jenkins

Nginx

Datadog

Git

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

10 GitHub Repos That Will Get You Hired in DevOps

What You Will Learn

These 10 GitHub repositories will help you build the core DevOps skills that employers look for. Each repo teaches a real skill, gives hands-on practice, and helps you build a strong portfolio.

- Linux and Troubleshooting
- Docker and Application Packaging
- CI/CD and Automation
- Terraform and Cloud Infrastructure
- Kubernetes and Deployments
- Monitoring, Reliability and Portfolio Projects

GitHub to Production Engineering Roadmap

GitHub — Your learning and portfolio starting point

→ **Code** — Clone, study and practice real projects

→ **CI/CD** — Automate testing and builds

→ **Container** — Package applications with Docker

→ **Deploy** — Deploy to Kubernetes

→ **Cloud** — Provision infrastructure with Terraform

→ **Monitor** — Observe, troubleshoot and improve

→ **Production** — Build job-ready skills and portfolio projects

The 10 Repositories and What They Teach

1. **DevOps Exercises** — Linux, networking, troubleshooting and interview practice.
2. **Awesome Compose** — Multi-container applications with Docker Compose.
3. **Starter Workflows** — CI/CD pipelines with GitHub Actions.
4. **Terraform Samples** — Infrastructure as Code on AWS.
5. **Kubernetes Examples** — Deploy and manage applications on Kubernetes.
6. **Minikube** — Run a local Kubernetes cluster.
7. **Argo CD** — GitOps and continuous deployment.
8. **Alertmanager** — Alerting, monitoring and incident response.
9. **Google Online Boutique** — Real-world microservices application.
10. **Real DevOps Projects** — End-to-end projects that combine multiple tools.

Outcome

- Practical DevOps skills
- Real GitHub portfolio projects
- Evidence for interviews
- Confidence to apply for junior DevOps roles

Remember

You don't just clone these repos. You study them, run them, troubleshoet hd them, make improvements and document what you learn.

Follow @veriqta for more DevOps learning

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Stop Collecting Repos. Start Building Evidence.

A Fork Is Not Enough

- Forking a repository only shows that you copied someone else's work.
- It does not show what you understand.
- Employers want proof that you can think, configure, troubleshoot and build.

What Employers Look For

- Practical skills, not just theory.
- Real examples of work you did.
- Configuration changes and testing.
- Evidence that you can troubleshoot.
- Clear documentation.
- Engineering thinking and decision making.

Show Your Work

- Make meaningful changes to the project.
- Add or update configuration files.
- Write and run tests.
- Verify that it works and show the results.
- Explain why you made each change.

Record Failures and Fixes

- Document what failed.
- Show the error message.
- Explain how you investigated.
- Describe the fix you applied.
- Share what you learned.

Document Your Decisions

- Explain why you chose a specific tool or configuration.
- Describe alternatives you considered.
- Mention trade-offs (cost, security, simplicity, scalability).
- Show that you can make thoughtful engineering decisions.

Show What You Implemented

- Clearly state what you built or changed.
- Highlight the exact files you modified.
- Include screenshots or command output where relevant.
- Explain how to run and verify your work.
- Be honest about what you learned from examples vs what you created yourself.

The 5 Things to Document for Every Project

1 What I Built

- Project overview
- What it does
- Tools used
- Link to repository

2 Changes I Made

- Files modified
- Configuration updates
- New features or improvements
- Why I made the changes

3 Tests and Verification

- How I tested it
- Test results
- Screenshots or command output
- What worked and what didn't

4 Failures and Fixes

- Error messages
- Root cause analysis
- Steps to fix
- What I learned
- How to prevent it in the future

5 My Takeaways

- Key skills learned
- Challenges faced
- Engineering decisions made
- How this project prepares me for real DevOps work

Turn learning into evidence. Evidence leads to opportunities.

VERIQTA

Repo 01: DevOps Exercises

Linux, networking, troubleshooting, interviews

Repository

bregman-arie/ devops-exercises

A collection of practical DevOps exercises to help you improve your Linux, networking, cloud and DevOps skills through hands-on challenges.

What You Will Learn

- Linux command line and system administration
- Networking, DNS, ports and connectivity
- Troubleshooting real-world problems
- Cloud and infrastructure fundamentals
- DevOps tooling and workflows
- Interview-style questions and scenarios

Why This Repo Is Valuable

- Hands-on exercises instead of theory only
- Helps you identify your knowledge gaps
- Covers real DevOps and infrastructure topics
- Builds troubleshooting and problem-solving skills
- Useful for technical interviews

Practice Lab

Investigate a failed network connection, explain the evidence, and record the diagnostic process.

Scenario

You cannot reach a web server at 93.184.216.34 (port 80). Use Linux commands to find the reason and fix the issue.

Network Troubleshooting Workflow



1. Check connectivity
2. Check DNS resolution
3. Check open ports
4. Check firewall rules
5. Find the cause and fix

Useful Commands

1. `ping 93.184.216.34` — Check basic connectivity
2. `nslookup example.com` — Check DNS resolution
3. `curl -I http://93.184.216.34` — Test HTTP connection
4. `nc -zv 93.184.216.34 80` — Check if port 80 is open
5. `traceroute 93.184.216.34` — Trace the network path

Example Output (Failure)

```
$ ping 93.184.216.34
--- 93.184.216.34 ping statistics ---
100% packet loss

$ curl -I http://93.184.216.34
curl: (7) Failed to connect to 93.184.216.34 port 80: Connection refused

$ nc -zv 93.184.216.34 80
nc: connect to 93.184.216.34 port 80 (tcp) failed: Connection refused
```

Troubleshooting Steps

1. Verify your local network connection.
2. Confirm the IP address is correct.
3. Check if the server is reachable from the internet.
4. Verify the port is open.
5. Check local firewall rules.
6. Look for DNS issues.
7. Compare with a working server.
8. Apply the fix and test again.

Career Value

- Demonstrates systematic troubleshooting.
- More than just memorizing commands.
- Shows you can investigate, find the root cause and explain your steps.
- These are the exact skills employers look for in junior DevOps engineers.

Your Task

- Complete one exercise from the repository.
- Document the problem, troubleshooting process and solution in your own words.
- Push your notes to a GitHub repository.
- Add screenshots of the commands and output.

VERIQTA

Repo 02: Docker Awesome Compose

Containers, services, networking, volumes

Repository

docker/ awesome-compose

A collection of real-world Docker Compose examples showing how multiple services work together. It includes common application stacks such as databases, web apps, monitoring tools and more.

What You Will Learn

- How multiple containers work together
- Docker Compose file structure
- Services, networks and volumes
- Environment variables and configuration
- Application dependencies and startup order
- Real-world multi-container applications

Why This Repo Is Valuable

- Shows practical container setups used by engineers in real projects.
- Helps you understand how services communicate over a network.
- Teaches production-like application stacks in a simple and practical way.
- Builds hands-on Docker and infrastructure skills.

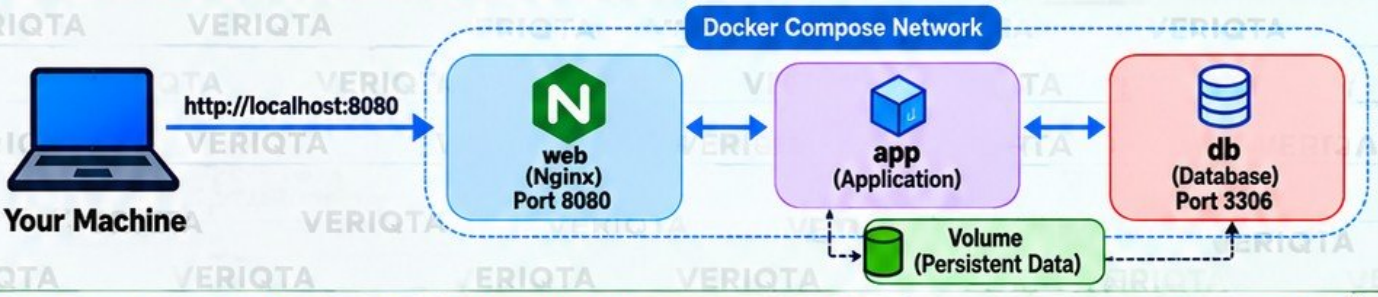
Practice Lab

- Run a multi-container application.
- Inspect the running services.
- Test connectivity between containers.
- Diagnose an intentionally stopped container.
- Make a small configuration change and verify it.

Important Note

These examples are for learning and local development. They are not production-ready as-is. Do not use them in production without proper hardening and security review.

How It Works (Example: Web App + Database)



Useful Commands

1. **docker compose up -d**
   Start all services in background
2. **docker compose ps**
   See running containers
3. **docker compose logs -f**
   View real-time logs
4. **docker compose down**
   Stop and remove containers
5. **docker compose exec web bash**
   Access a running container

Example: docker-compose.yml

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    depends_on:
      - app
  app:
    image: python:3.11-slim
    command: python app.py
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: example
    volumrk:
      - db_data:/var/lib/postgresql/data
volumes:
   db_data:
```

Troubleshooting Example

Problem: The app container cannot connect to the database.

1. Check container status
   `docker compose ps`
2. View logs
   `docker compose logs app`
3. Test connectivity from inside container
   `docker compose exec app ping db`
4. Common causes
   - Wrong service name
   - Container not running
   - Network or configuration issue
6. Fix the configuration and restart
   `docker compose up -d`

Career Value

- Demonstrates practical container operations.
- Shows you can work with multi-service applications.
- Helps you understand application architescture.
- A key skill for DevOps, platform and cloud roles.

Your Task

- Choose one example from the repository.
- Run it locally and explore the services.
- Intentionally stop one container and troubleshoot it.
- Document what you learned and add it to your GitHub.

VERIQTA

Repo 03: GitHub Actions Starter Workflows

Continuous integration, testing, pipeline automation

Repository

actions/starter-workflows

A collection of ready-to-use GitHub Actions workflow templates for many languages and platforms. It helps you quickly set up CI workflows to build, test and automate your projects.

What You Will Learn

- How GitHub Actions works
- Workflow file structure (YAML)
- Triggers (push, pull request, etc.)
- Running builds and tests automatically
- Using actions for real-world tasks
- Debugging failed workflows
- Adapting templates for your own projects

Why This Repo Is Valuable

- Gives real, working workflow examples
- Shows industry best practices
- Helps you implement automated testing
- Useful for any DevOps or software engineer role
- Teaches CI concepts through practical templates

Practice Lab

1. Choose a starter workflow (e.g. Python, Node.js)
2. Add it to your own repository
3. Run it on a pull request
4. Intentionally introduce a failing test
5. Check the workflow logs and fix the issue

How It Works

1. You push code or create a pull request.
2. GitHub Actions runs the workflow automatically.
3. It executes the defined jobs (build, test, etc.).
4. You get the results directly in GitHub.

GitHub Actions Workflow Flow

1. **Code Change** (Push or Pull Request)
2. **GitHub** Triggers Workflow
3. **Jobs Run** (Build, Test, etc.)
4. **Check Results** (Pass or Fail)
5. **View Logs** and Fix Issues

Example Workflow (Python)

```yaml
name: Python CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest
```

Test Failure Example

```
==================== test session starts ====================
FAILED test_app.py::test_add
E   assert 1 == 2
E    +  where 1 = add(1, 1)
==================== short test summary ====================
FAILED test_app.py::test_add - assert 1 == 2
1 failed, 2 passed in 0.45s
```

Troubleshooting Steps

1. Read the error message carefully.
2. Check the test code and expected result.
3. Fix the code or test.
4. Push the changes and check the workflow again.

Career Value

- Demonstrates ability to implement CI pipelines.
- Shows you can write and debug automated tests.
- Proves you understand real DevOps workflows.
- Employers look for engineers who can maintain and improve CI/CD pipelines.

Important Note

- These are example workflows for learning and development.
- They are not production-ready as-is.
- Review and customize them for your own use cases and security needs.

VERIQTA

Repo 04: Terraform AWS Samples

Infrastructure as Code, AWS, security, cost

Repository

build-on-aws/ terraform-samples

A collection of Terraform configuration examples for different AWS services. It shows how to define and deploy real infrastructure using version-controlled configuration files.

What You Will Learn

- How Terraform works with AWS
- Infrastructure as Code principles
- AWS services (VPC, EC2, S3, RDS, etc.)
- Configuration structure and modules
- Managing environment variables and state
- Reviewing changes before deployment
- Security, cost and best practices

Why This Repo Is Valuable

- Real-world examples for multiple AWS services
- Shows practical and production-style structure
- Helps you understand infrastructure design
- Teaches best practices for security and cost
- Useful for DevOps, cloud and SRE roles
- Helps you learn by example and adapt to build your own projects

Practice Lab

1. Inspect a Terraform sample from the repository
2. Initialize the configuration
3. Validate the Terraform files
4. Run a plan and review the proposed changes
5. Identify potential costs and security risks
6. (Optional) Deploy to a safe environment and verify

Important Note

- These examples are for learning and development only.
- They can create real AWS resources which may incur cost.
- Review all configuration before deploying.
- Use cost estimates and clean up resources after testing.

How Terraform Works with AWS

1. Define Infrastructure (Terraform Files)
2. Initialize (Download Providers)
3. Plan (Preview Changes)
4. Apply (Create Resources in AWS)
5. Manage (Updates and Version Control)

Example: S3 Bucket (main.tf)

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"

  tags = {
    Name        = "veriqta-demo"
    Environment = "dev"
  }
}
```

Common Useful Commands

1. **terraform init** — Download providers and modules
2. **terraform validate** — Check configuration syntax
3. **terraform plan** — Preview changes
4. **terraform apply** — Create resources (after review)
5. **terraform destroy** — Remove resources (cleanup)

Reviewing a Plan

```
$ terraform plan
Terraform will perform the
following actions:

  # aws_s3_bucket.example will be
    created
  + resource "aws_s3_bucket"
      "example" {
      + bucket = "veriqta-demo-bucket"
      + tags   = {
          + Environment = "dev"
          + Name        = "veriqta-demo"
        }
    }
Plan: 1 to add, 0 to change, 0 to destroy.
```

Career Value

- Demonstrates practical infrastructure as code skills.
- Shows understanding of controlled infrastructure changes.
- Builds awareness of cloud cost and security considerations.
- Valuable for DevOps, cloud and platform engineering roles.

Security and Cost Considerations

- Use least privilege IAM permissions.
- Review resource configuration and public access.
- Check estimated cost before applying.
- Delete resources after testing.

VERIQTA

Repo 05: Kubernetes Examples

Pods, Deployments, Services, application configuration

Repository

kubernetes/examples

A collection of Kubernetes configuration examples that show how to deploy applications using Pods, Deployments, Services and other resources. It helps you learn real Kubernetes concepts through practical files.

What You Will Learn

- Kubernetes core concepts and architecture
- Pods, Deployments, Services, ConfigMaps, Secrets and more
- How declarative configuration works
- Application deployment and exposure
- Troubleshooting common issues
- Real-world Kubernetes manifests and examples

Why This Repo Is Valuable

- Real Kubernetes manifests you can study and run
- Shows best practices for application deployment
- Helps you understand how Kubernetes maintains desired state
- Covers multiple use cases and configuration types
- Useful for DevOps, cloud and platform engineering roles

Practice Lab

1. Deploy a simple application from the examples
2. Inspect the running Pods
3. Expose the application using a Service
4. Intentionally use an incorrect image reference
5. Observe the error and troubleshoot the issue
6. Fix the configuration and verify it works

How Kubernetes Maintains Application Workloads



Example: Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

Useful Commands

1. `kubectl apply -f deployment.yaml` — Create the resources
2. `kubectl get pods` — Check running Pods
3. `kubectl get svc` — View Services
4. `kubectl describe pod <pod-name>` — See detailed information
5. `kubectl logs <pod-name>` — View application logs
6. `kubectl delete -f deployment.yaml` — Remove the resources

Intentional Error Example

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: nginx
        image: nginx:invalid-tag
        ports:
          - containerPort: 80
```

Expected Result

- Pod goes into ImagePullBackOff or ErrImagePull
- Check the pod description and events
- Identify the invalid image reference
- Fix the image name and redeploy

Career Value

- Demonstrates foundational Kubernetes deployment and troubleshooting skills.
- Shows you can work with real Kubernetes resources.
- Builds practical experience with declarative configuration.
- Useful for DevOps, SRE and platform engineering roles.

Key Takeaways

- Kubernetes uses declarative configuration to maintain the desired state.
- Deployments manage Pods and ensure availability.
- Services provide stable access to applications.
- You can troubleshoot issues by checking pod status, logs and events.
- These examples help you learn by doing.

VERIQTA

Repo 06: Minikube

Local Kubernetes clusters and cluster operations

Repository

kubernetes/minikube

Minikube is a tool for running a local Kubernetes cluster on your machine. It helps you learn and experiment with Kubernetes without needing a cloud environment.

What You Will Learn

- What Minikube is and when to use it
- Difference between running an application in Kubernetes and operating the cluster
- Cluster components (nodes, control plane, addons)
- Managing and inspecting cluster resources
- Deploying and exposing applications
- Troubleshooting scheduling and resource issues
- Useful commands for day-to-day cluster operations

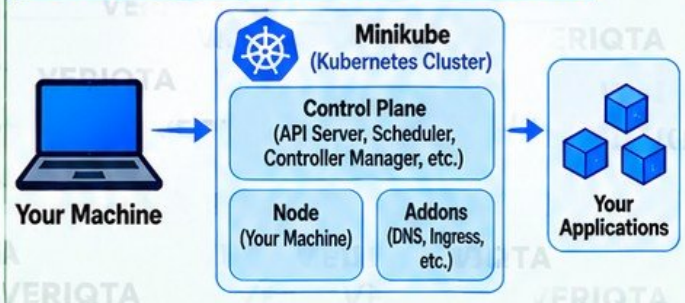
Why This Repo Is Valuable

- Gives you a real local Kubernetes environment
- Helps you understand how the cluster works behind your application
- Lets you practice common operational tasks
- Builds troubleshooting and diagnostic skills
- Useful for DevOps, cloud and SRE roles
- Safe environment to learn before using managed Kubernetes services in the cloud

Practice Lab

1. Create a local cluster using Minikube
2. Inspect the cluster and nodes
3. Deploy a test application
4. Expose the application and access it
5. Investigate a scheduling problem (e.g. insufficient resources)
6. Fix the issue and verify the application runs

Minikube Architecture (Simplified)



Important Note

- Minikube is for learning and development only.
- It is not a production cluster.
- It runs on your local machine and uses limited resources.
- Some features may be different from managed Kubernetes services (EKS, GKE, AKS).

Common Minikube Commands

1. `minikube start`
2. `minikube status`
3. `minikube nodes`
4. `minikube addons list`
5. `kubectl get nodes`
5. View nodes using kubectl
6. Open Kubernetes dashboard
7. Stop the cluster
8. Delete the cluster

Example: Deploy a Test Application

```
# Start Minikube
minikube start

# Deploy a sample application
kubectl create deployment nginx --image=nginx
kubectl get pods

# Expose the application
kubectl expose deployment nginx --type=NodePort \
 --port=80
kubectl get svc

# Access the application
minikube service nginx
```

Scheduling Problem Example

```
# Deploy a pod with high resource request
kubectl apply -f high-cpu-pod.yaml
kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
high-cpu      0/1     Pending   0          1m
```

Check the Reason

```
kubectl describe pod high-cpu
Events:
  Type     Reason            Message
  ----     ------            -------
  Warning  FailedScheduling  Insufficient cpu
```

Career Value

- Builds familiarity with cluster state and capacity.
- Teaches you cluster operations and troubleshooting.
- Shows you can work with real Kubernetes tools and commands.
- Helps you understand the difference between application configuration and cluster management.
- A key skill for DevOps, SRE and platform engineering roles.

Key Takeaways

- Minikube helps you run a local Kubernetes cluster.
- You learn both application deployment and cluster operations.
- You can inspect, troubleshoot and fix common issues.
- It is a safe and practical way to build Kubernetes skills.
- These skills transfer to managed Kubernetes services in the cloud.

VERIQTA

Repo 07: Argo CD

GitOps, deployment reconciliation, configuration drift

Repository

argoproj/argo-cd

Argo CD is a declarative, GitOps continuous delivery tool for Kubernetes. It uses Git as the single source of truth for application deployments and automatically keeps your cluster in the desired state.

What You Will Learn

- What Argo CD is and how GitOps works
- How Git acts as the declared source of truth
- Argo CD architecture and key components
- Application, project and repository configuration
- Synchronization and deployment reconciliation
- Handling configuration drift
- Real-world use cases and best practices

Why This Repo Is Valuable

- Shows how Git can control Kubernetes deployments.
- Helps you understand modern GitOps practices.
- Demonstrates automatic synchronization and drift detection.
- Builds practical skills with deployment control and auditability.
- Useful for DevOps, platform and SRE roles.

Practice Lab

1. Study an Argo CD application example.
2. Review the Git repository structure and manifests.
3. Deploy Argo CD (using a local cluster).
4. Create an Application pointing to a Git repo.
5. Make a change in Git and watch Argo CD synchronize it to the cluster.
6. Intentionally introduce a drift and observe how Argo CD detects and reconciles it.

How Argo CD Works (GitOps Workflow)



Example: Argo CD Application Manifest

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/k8s-manifests.git
    targetRevision: main
    path: nginx
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Useful Commands

1. `kubectl get pods -n argocd` — Check Argo CD pods
2. `kubectl port-forward svc/argocd-server 8080:443 -n argocd` — Access Argo CD UI locally
3. `kubectl get applications -n argocd` — List Argo CD applications
4. `kubectl describe application <name>` — View application details
5. `kubectl logs -n argocd deploy/argocd-application-controller` — Check controller logs

Drift Detection Example

1. Change the live cluster manually (e.g. scale deployment).
2. Observe Argo CD UI showing **OutOfSync** status.
3. Check the diff to see what changed.
4. Click Sync or enable auto-sync to reconcile the drift.

Career Value

- Demonstrates understanding of GitOps and deployment control.
- Shows you can work with real-world continuous delivery tools.
- Introduces auditability and change tracking.
- Valuable for DevOps, platform and SRE roles.

Key Takeaways

- Git is the single source of truth for desired state.
- Argo CD continuously reconciles the cluster with Git.
- It detects and can automatically fix configuration drift.
- It provides visibility, auditability and safer deployments.
- These skills are used in modern cloud-native environments.

VERIQTA

Repo 08: Prometheus Alertmanager

Alert routing, grouping, silencing, incident response

Repository

prometheus/alertmanager

Alertmanager is a component of the Prometheus ecosystem that handles alert routing, grouping, silencing and notification delivery. It ensures the right alerts reach the right engineers at the right time.

What You Will Learn

- Why metrics alone are not enough
- How alerts are generated and sent to Alertmanager
- Alert routing, grouping and deduplication
- Silences and maintenance windows
- Notification channels (email, Slack, etc.)
- Handling real-world incident scenarios
- Best practices for alerting and on-call

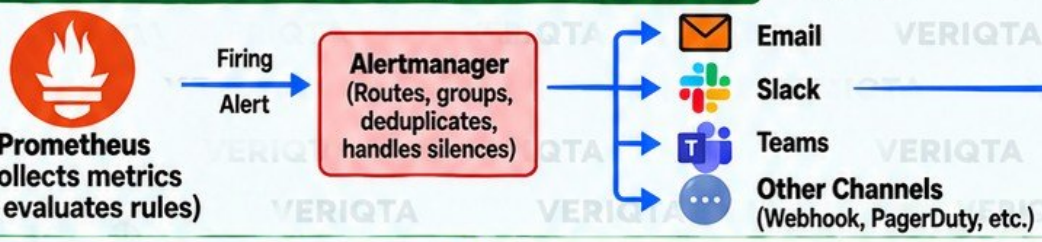
Why This Repo Is Valuable

- Teaches you how alerts are routed and managed in real systems.
- Shows grouping and deduplication to reduce alert noise.
- Helps you understand silencing and maintenance windows.
- Builds incident response and on-call awareness.
- Useful for DevOps, SRE and platform engineering roles.

Practice Lab

1. Explore the sample configuration files.
2. Run Prometheus and Alertmanager locally (using Docker or provided examples).
3. Trigger a test alert from Prometheus.
4. Trace how the alert is received, grouped and routed by Alertmanager.
5. Configure a silence and observe its effect.
6. Verify that duplicate alerts are grouped and not sent multiple times.

Alert Flow: From Metrics to Engineer Notification



Example: Alertmanager Configuration

```
global:
  resolve_timeout: 5m

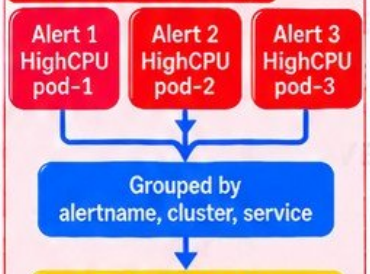
route:
  group_by: ["alertname", "cluster", "service"]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 3h
  receiver: "team-slack"

receivers:
- name: "team-slack"
  slack_configs:
    - api_url: "https://hooks.slack.com/services/XXX"
      channel: "#alerts"
      send_resolved: true
```

Sample Alert (from Prometheus)

```
groups:
- name: example-rules
  rules:
  - alert: HighCPUUsage
    expr: node_cpu_seconds_total{
    mode="idle"} < 0.2
    for: 1m
    labels:
      severity: warning
    annotations:
      summary: "High CPU usage"
      description: "CPU usage is
      above 80% for more than 1 minute."
```

Grouping Example



Career Value

- Demonstrates understanding of observability and alerting.
- Shows ability to configure and manage alert pipelines.
- Highlights incident response and on-call thinking.
- Builds practical experience with real DevOps and SRE tools.
- Valuable for DevOps, SRE and platform engineering roles.

Key Takeaways

- Metrics show you what is happening, but alerts tell you when you need to act.
- Alertmanager groups and deduplicates alerts to reduce noise.
- You can route, silence and manage alerts for different teams.
- Proper alerting leads to faster incident response.
- These skills are essential in real production environments.

VERIQTA

Repo 09: Google Online Boutique

Microservices, distributed systems, service dependencies

Repository

GoogleCloudPlatform/microservices-demo

Google Online Boutique is a realistic e-commerce application composed of multiple microservices. It demonstrates how different services work together to provide a complete user experience in a distributed system.

What You Will Learn

- What Google Online Boutique is and its architecture
- How multiple services cooperate to handle user requests
- Service dependencies and communication
- How failures in one service affect the user journey
- Observability, tracing and monitoring concepts
- Real-world microservices patterns and best practices

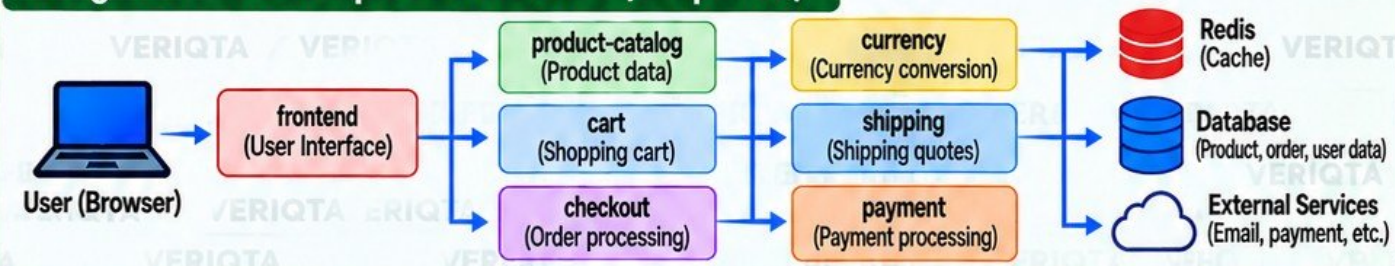
Why This Repo Is Valuable

- Provides a real-world, production-like e-commerce application.
- Helps you understand microservices architecture and service dependencies.
- Lets you practice troubleshooting distributed systems.
- Demonstrates modern cloud-native technologies (Kubernetes, gRPC, Docker, observability).
- Useful for DevOps, SRE and platform engineering roles.

Practice Lab

1. Run the Google Online Boutique application locally (using Docker or Kubernetes).
2. Explore the different services and their roles.
3. Map the frontend, cart, checkout and supporting services.
4. Simulate a failure in one service and observe the impact on the user experience.
5. Investigate logs and traces to identify the issue.
6. Restore the service and verify the application works again.

Google Online Boutique Architecture (Simplified)



Example: Run with Docker

```
# Clone the repository
git clone https://github.com/GoogleCloudPlatform/microservices-demo.git
cd microservices-demo

# Run using Docker Compose
docker compose up -d

# Check running services
docker compose ps

# Access the application
http://localhost:8080
```

Common Useful Commands

1. `docker compose ps` — View running services
2. `docker compose logs -f <service>` — View service logs
3. `docker compose restart <service>` — Restart a service
4. `docker compose down` — Stop all services
5. `docker compose up -d` — Start all services
6. `docker compose logs -f frontend` — Check frontend logs

Example: Service Failure

```
# Stop the cart service
docker compose stop cart

# Try to add item to cart in the UI (You will see an error)

# Check the logs
docker compose logs -f cart

# Restart the service
docker compose start cart

# Verify the application works again
```

Career Value

- Demonstrates understanding of microservices architecture.
- Shows how service dependencies affect real applications.
- Builds troubleshooiting and debugging skills in distributed systems.
- Introduces observability and operational thinking.
- Valuable for DevOps, SRE and cloud engineering roles.

Key Takeaways

- A real application uses many cooperating services.
- Failure in one service can affect the entire user journey.
- Understanding dependencies helps you troubleshoot faster.
- Observability tools (logs, traces, metrics) are essential.
- These skills are directly applicable to real production systems.

VERIQTA

Repo 10: Real-World DevOps Projects

Cloud architecture, automation, delivery, integrated projects

Repository

techiescamp/devops-projects

A collection of real-world DevOps projects that show how infrastructure, security, deployment, monitoring and automation work together. The projects cover multiple cloud providers and modern DevOps tools with step-by-step documentation.

What You Will Learn

- How real-world DevOps projects are designed
- Cloud architecture and infrastructure components
- Infrastructure as Code, automation and CI/CD
- Security considerations (IAM, networking, secrets)
- Deployment and environment management
- Monitoring, logging and incident visibility
- Cost management and cleanup
- Best practices for integrated DevOps engineering

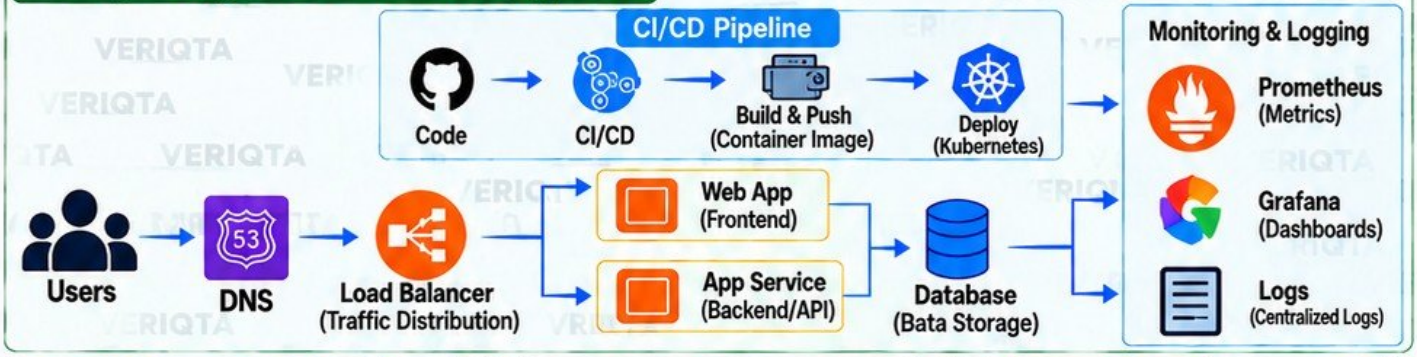
Why This Repo Is Valuable

- Brings multiple DevOps tools and services together in complete projects.
- Shows how infrastructure, security, deployment and monitoring work as one system.
- Helps you understand real-world architecture and environment setup.
- Builds practical, end-to-end DevOps engineering skills.
- Useful for DevOps, cloud and platform engineering roles.

Practice Lab

1. Explore the available projects in the repository.
2. Choose a project and read its documentation.
3. Identify the architecture, tools and services used.
4. Check the prerequisites and estimated cost.
5. Follow the deployment steps in a safe environment (e.g. cloud free tier).
6. Test the application and verify monitoring.
7. Clean up all resources and confirm no leftover costs.

Example Project Architecture (Typical)



Example: Clone and Explore

```
# Clone the repository
git clone https://github.com/techiescamp/
devops-projects.git
cd devops-projects

# List available projects
ls

# Open a specific project folder
cd project-name

# Read the documentation
cat README.md
```

Key Things to Check in Each Project

1. Architecture diagram and components
2. Prerequisites (tools, accounts, access)
3. Estimated cost (free tier or paid)
4. Infrastructure setup (IaC)
5. Deployment steps (CI/CD, containers, etc.)
6. Security considerations (IAM, networking)
7. Monitoring and logging setup
8. Cleanup steps to remove all resources

Career Value

- Demonstrates ability to work on end-to-end DevOps projects.
- Shows understanding of real-world architecture and service integration.
- Builds experience with cloud, automation, deployment, security and monitoring.
- Helps you move from isolated tool exercises to integrated DevOps engineering work.
- Valuable for DevOps, SRE, cloud and platform engineering roles.

Key Takeaways

- Real-world projects combine infrastructure, security, deployment and monitoring.
- Always understand the architecture, prerequisites and cost before deploying.
- Test the application and monitoring after deployment.
- Clean up all resources to avoid unexpected charges.
- These projects build the practical skills needed for real DevOps and cloud engineering roles.

VERIQTA

Build Your First Original DevOps Project

From following examples to building independently

Project Overview

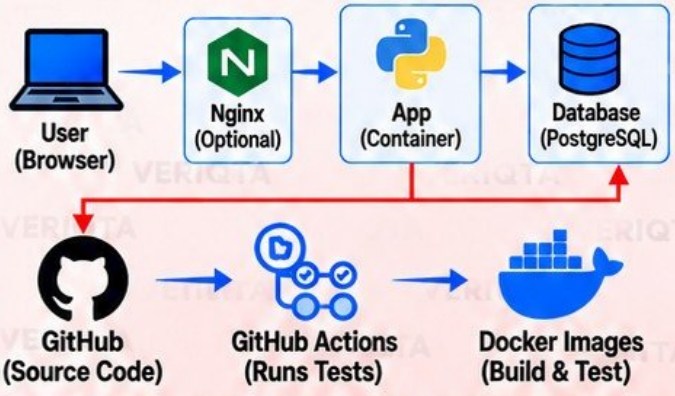
In this project, you will build your own simple application, containerize it, add a Compose configuration, set up a CI workflow, and document how to run and verify it.

This helps you move from following examples to building independently.

What You Will Build

- A simple web application (e.g. a to-do app)
- A Dockerfile to containerize the application
- A Docker Compose configuration (app + database)
- A CI workflow that runs tests on every push and pull request
- Clear documentation on how to run and verify the application
- A complete, original repository that you built

Project Architecture (Example)



Step-by-Step Tasks

1. Choose a simple application (e.g. to-do app with Python/Node.js).
2. Create the application code.
3. Add a Dockerfile to containerize the app.
4. Write a docker-compose.yml with app and database.
5. Add a GitHub Actions workflow to run tests.
6. Document how to run and verify the application.
7. Push your project to GitHub.

Example: Dockerfile (Python)

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir \
    -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

Example: docker-compose.yml

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: todoapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db-data:/var/lib/postgresql/data
volumes:
  db-data:
```

Example: GitHub Actions (CI)

```
name: CI
on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest
```

How to Run and Verify

1. Clone your repository from GitHub.
2. Run `docker compose up --build`.
3. Open the application in your browser (e.g. http://localhost:8000).
4. Verify that the app works and the database is connected.
5. Make a code change, push it, and check the CI workflow.

Deliverable

An original containerized application repository with:

- Application code
- Dockerfile
- docker-compose.yml
- GitHub Actions workflow
- README with setup and verification instructions

Career Value

- Builds confidence to work independently.
- Shows ability to integrate multiple DevOps tools.
- Demonstrates real project structure and documentation.
- Helps you move from isolated exercises to end-to-end engineering work.
- Valuable for DevOps, SRE and platform engineering roles.

Key Takeaways

- Start simple, keep the scope small.
- Focus on a working, well-documented project.
- Combine containerization, automation and testing.
- This is the foundation for more advanced real-world projects.

Follow @veriqta for more DevOps learning

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Make Your Project Production-Aware

From a working demo to an operationally responsible application

What This Page Is About

A working demo is a good start, but a production-aware application is more reliable, secure and maintainable. Here you will improve your project by adding essential operational features that are used in real engineering environments.

Key Concepts You Will Implement

1. Health checks and application logs
2. Environment-specific configuration
3. Secret handling and least privilege
4. Resource limits and failure recovery
5. Clear documentation and verification

Demo vs Production-Aware

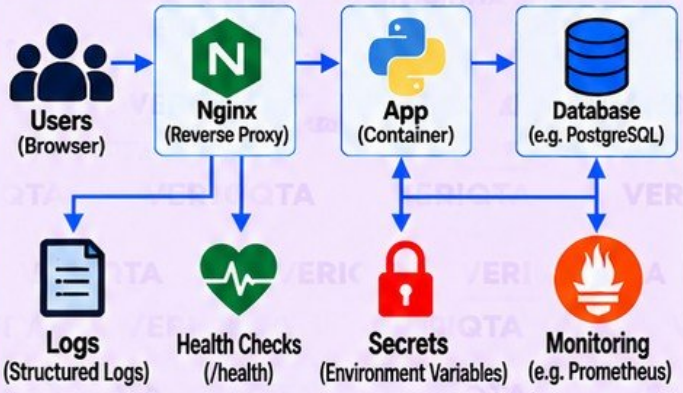
Working Demo

- ✗ Runs locally
- ✗ Basic configuration
- ✗ No health checks
- ✗ No proper logging
- ✗ Hardcoded settings
- ✗ No resource limits
- ✗ Not ready for real use

Production-Aware

- ✓ Reliable and observable
- ✓ Health checks enabled
- ✓ Structured application logs
- ✓ Environment-specific config
- ✓ Secrets handled securely
- ✓ Resource limits configured
- ✓ Recovers from failures
- ✓ Ready for real environments

Project Architecture (Improved)



1. Health Checks

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r \
    requirements.txt

HEALTHCHECK --interval=30s \
  --timeout=5s --retries=3
   CMD curl -f \
   http://localhost:8000/health \
   || exit 1
```

- Add a /health endpoint
- Configure container health check
- Verify the application is healthy

2. Application Logs

```
#  app.py
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s \
       %(levelname)s
       %(message)s"
)

@app.get("/health")
def health():
    logging.info("Health
     check called")
    return {"status": "ok"}
```

- Use structured logging
- Log important events
- Check logs using docker logs

3. Environment Config

```
# docker-compose.yml
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - APP_ENV=production
      - LOG_LEVEL=info
      - DATABASE_URL=
        postgres://db:5432/app
    env_file:
      - .env
  db:
    image: postgres:15
    env_file:
      - .env
```

- Use .env files
- Different config for dev and prod
- Avoid hardcoding values

4. Secrets & Least Privilege

```
# Example (Docker Compose)
services:
  app:
    image: myapp:latest
    environment:
      - DATABASE_URL_FILE=
        /run/secrets/db_url
    secrets:
      - db_url
    user: "1000:1000"
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
secrets:
  db_url:
    file: ./secrets/db_url.txt
```

- Use secrets (not hardcoded)
- Run as non-root user
- Use least privileges and read-only filesystem

5. Resource Limits & Recovery

```
# docker-compose.yml
services:
  app:
    build: .
    deploy:
      resources:
        limits:
          cpus: "0.50"
          memory: "512M"
        reservations:
          cpus: "0.25"
          memory: "256M"
    restart: unless-stopped
```

- Set CPU and memory limits
- Configure restart policy
- Test what happens when the app fails
- Verify the application recovers automatically

Deliverable

A project improvement checklist with verified changes:

- Health check implemented
- Application logs added
- Environment-specific configuration
- Secrets handled securely
- Resource limits configured
- Failure recovery tested
- Documentation updated
- All changes verified

Career Value

- Shows understanding of real-world production requirements.
- Demonstrates operational thinking and responsibility.
- Builds skills in reliability, security and maintainability.
- Helps you stand out in DevOps, SRE and platform engineering roles.

Key Takeaways

- A working application is just the beginning.
- Production-aware applications are reliable, secure and observable.
- These practices are essential in real engineering environments.

Follow @veriqta for more DevOps learning

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Page 15

Troubleshoot Three Failures Like an Engineer

VERIQTA

Use evidence to find the root cause, apply a fix and verify the solution

Page Overview

In this page, you will troubleshoot three common DevOps failures using a clear, evidence-based process. You will learn how to gather the right information, identify the root cause, apply a fix and verify that the issue is resolved.

What You Will Learn

1. A structured debugging process
2. How to read and use logs, events and error messages
3. How to identify the real root cause
4. How to apply a correct fix
5. How to verify that the issue is resolved
6. How to document an incident like an engineer

The Debugging Process

1. Identify the problem (symptoms)
2. Gather evidence (logs, events, configs)
3. Find the root cause
4. Apply a fix
5. Verify the solution
6. Record the incident (what happened and what you did)

Incident Record Template

- Incident Title:
- Symptoms:
- Evidence (logs, errors, events):
- Root Cause:
- Correction (fix):
- Verification (how you confirmed):

1. Container Exits Immediately

Scenario

A container starts and exits immediately with a non-zero code.

```
$ docker ps -a
ae3bcd1   myapp   "python app.py"
Exited (1) 5 seconds ago

$ docker logs ae3bcd1
Traceback (most recent call last):
  File "app.py", line 1, in <module>
    import flask
ModuleNotFoundError: No module named 'flask'
```

Fix

- Install missing dependencies in the Dockerfile.
- Rebuild the image and run the container again.
- Verify the application starts and stays running.

2. CI Pipeline Fails After a Commit

Scenario

A code change is pushed and the CI pipeline fails during the test job.

```
$ Run pytest
================== FAILURES ==============
E   ModuleNotFoundError: No module named 'app'

ERROR: tests/test_app.py::test_home -
ModuleNotFoundError
```

Fix

- Review the test output and identify the error.
- Check recent code changes and dependencies.
- Correct the import path or update requirements.
- Push the fix and verify the pipeline passes.

3. Kubernetes Pod Cannot Start

Scenario

A Pod remains in a non-running state (e.g. ImagePullBackOff or CrashLoopBackOff).

```
$ kubectl get pods
NAME          READY   STATUS
web-7d9f8c6f  0/1     ImagePullBackOff

$ kubectl describe pod web-7d9f8c6f
Events:
  Type     Reason   Message
  ----     ------   -------
  Warning  Failed   Failed to pull image
                    "myapp:latest":
                    image not found
```

Fix

- Check the image name and registry.
- Correct the image reference or push the image.
- Apply the change and verify the Pod is running.

Example Incident Record (Container)

Title: Container exits immediately
Symptoms: Container stops with exit code 1
Evidence: docker logs shows ModuleNotFoundError
Root Cause: Missing Python dependency
Correction: Add flask to requirements.txt and rebuild
Verification: Container stays running and app responds with 200 OK

Example Incident Record (CI Pipeline)

Title: CI pipeline fails after a commit
Symptoms: Test job fails
Evidence: ModuleNotFoundError in test output
Root Cause: Incorrect import path
Correction: Fix import and update requirements
Verification: Pipeline passes successfully

Example Incident Record (Kubernetes)

Title: Pod cannot start
Symptoms: ImagePullBackOff
Evidence: kubectl describe shows image not found
Root Cause: Incorrect image name
Correction: Update image tag and redeploy
Verification: Pod reaches Running state

Deliverable

A short incident record for each failure containing:

- Symptoms
- Evidence
- Root cause
- Correction
- Verification

Key Takeaways

- Use a structured, evidence-based process.
- Logs, events and error messages tell the story.
- Always verify that your fix actually resolves the issue.
- Document what happened and what you did.
- These troubleshooting skills are essential in real production environments.

Career Value

- Builds real-world troubleshooting skills.
- Shows ability to analyse problems using evidence.
- Develops a systematic and calm approach.
- Useful for DevOps, SRE and platform engineering roles.

VERIQTA

Build a GitHub Portfolio Recruiters Can Understand

Present your engineering work clearly and professionally

Why a Good Portfolio Matters

- Helps recruiters quickly understand what you can do.
- Shows real, practical engineering skills.
- Makes it easy to run and verify your work.
- Demonstrates clear communication.
- Increases your chances of interviews.

Key Elements of a Strong Repository

1. Descriptive repository name
2. Clear README with prerequisites and reproducible setup
3. Architecture diagrams and screenshots
4. Tests, limitations, cleanup and security considerations
5. Clear separation of original work from borrowed examples

Recommended Repository Structure

```
my-devops-project
├── app/          Application source code
├── docker/       Dockerfiles and related files
├── k8s/          Kubernetes manifests (if used)
├── .github/      CI workflow files
├── docs/         Architecture diagrams, screenshots
├── tests/        Automated tests
├── README.md     Setup, usage, explanation
├── LICENSE       License file
└── .gitignore    Ignore unnecessary files
```

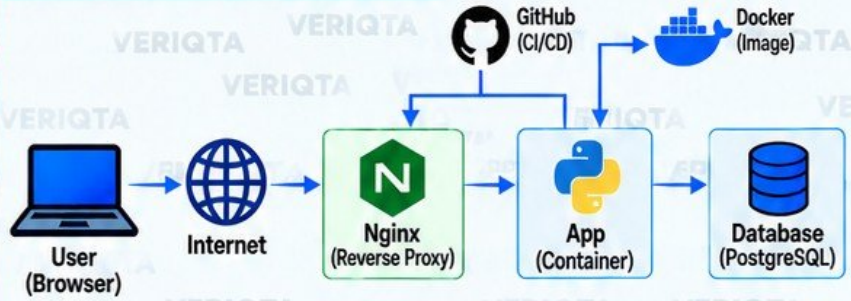
README.md Should Include

1. Project overview and objectives
2. Features and what the project does
3. Prerequisites (tools, accounts, access)
4. Step-by-step setup instructions
5. How to run and verify the application
6. Architecture diagram
7. Screenshots or demo output
8. How to run tests
9. Known limitations
10. Cleanup steps
11. Security considerations
12. Original work vs borrowed examples

Example README Sections

- **Project Overview** — What it does and why you built it
- **Prerequisites** — Tools, versions and accounts needed
- **Setup and Run** — Step-by-step commands
- **Architecture** — Diagram of how it works
- **Screenshots** — UI, logs or demo output
- **Tests** — How to run tests and expected results
- **Limitations** — What is not included or known issues
- **Cleanup** — How to remove resources
- **Security Considerations** — Secrets, access and best practices

Example Architecture Diagram



Example Screenshots (Demo)



Example Tests Section

```
$ docker compose run --rm app pytest
================ test session starts ============
collected 3 items
tests/test_app.py ...                    [100%]
================ 3 passed in 1.25s ==============
```

Deliverable

One professionally documented portfolio repository containing:

- Working application code
- Clear README
- Architecture diagram
- Tests and verification steps
- Screenshots
- Security and cleanup instructions

Career Value

- Makes your work easy for recruiters to understand and run.
- Demonstrates both technical and communication skills.
- Shows real-world engineering practices.
- Differentiates your original work from copied examples.
- Increases your chances of interviews and job offers.

Key Takeaways

- Present your projects clearly and professionally.
- Make it easy for anyone to run and verify your work.
- Good documentation is a key engineering skill.
- A strong GitHub portfolio can open real opportunities.

VERIQTA

Turn Your Project into Resume Evidence

Translate your engineering work into clear, credible resume statements

Why This Matters

- Helps you present your work clearly and professionally.
- Shows the value and impact of your projects.
- Makes it easy for recruiters to understand your skills.
- Increases your chances of getting interviews.
- Builds confidence in talking about your work.

Key Steps to Create Strong Resume Evidence

1. Describe what you built.
2. Identify the problem you solved.
3. Explain the tools and engineering decisions.
4. Include measured results only when verified.
5. Avoid invented production experience or exaggerated claims.
6. Write clear, concise and specific resume bullets.

From Project to Resume Bullet

1. Your Project (Repository)
2. Problem You Solved (Why it matters)
3. Tools & Engineering Decisions (How you built it)
4. Results (Verified only)
5. Resume Bullet (Clear and concise)

What to Include in a Resume Bullet

- What you built (application, tool, pipeline, etc.)
- The problem you solved
- Key tools and technologies used
- Important engineering decisions (e.g. why you chose a tool or architecture)
- Measurable results (only if verified)
- Your specific role and contribution

Example: From Project to Resume Bullet 1

Project: Containerized Web Application

Python Application → Docker (Container) → GitHub Actions (CI/CD) → Deployed (Cloud/Local)

Resume Bullet:

- Built and containerized a Python web application with Docker and Docker Compose, implemented automated testing using GitHub Actions, and created clear documentation and setup instructions to ensure reproducible deployment.

Example: From Project to Resume Bullet 2

Project: Kubernetes Deployment with Monitoring

Kubernetes (Deployment) → Prometheus (Monitoring) → Grafana (Visualization) → Logs (Observability)

Resume Bullet:

- Deployed a multi-service application on Kubernetes, configured monitoring with Prometheus and Grafana, set up health checks and resource limits, and documented the architecture and troubleshooting steps to demonstrate production-aware engineering practices.

Tips for Writing Credible Resume Bullets

- Be specific about what you built.
- Focus on the problem and your solution.
- Mention the exact tools and technologies used.
- Include results only if you actually measured them.
- Keep each bullet clear, concise and impactful.
- Avoid claiming real production experience if it was a learning project.
- Use action words like Built, Designed, Implemented, Automated, Improved.

Common Mistakes to Avoid

- Using vague statements (e.g. "worked on DevOps tools").
- Claiming you worked on large production systems.
- Exaggerating your role or impact.
- Listing tools without context.
- Including unverified or made-up metrics.
- Copying generic resume bullets from the internet.

Deliverable

Two accurate project-based resume bullets that clearly explain:

- What you built
- The problem you solved
- Tools and engineering decisions
- Verified results (if available)

Career Value

- Helps recruiters quickly understand your skills and experience.
- Demonstrates real engineering and problem-solving ability.
- Makes your resume stand out for DevOps, SRE and platform roles.
- Builds confidence for interviews.
- Shows you can translate technical work into business value.

VERIQTA

Explain Your GitHub Projects in Interviews

Discuss your work confidently with clear, real and evidence-based answers

Why This Matters

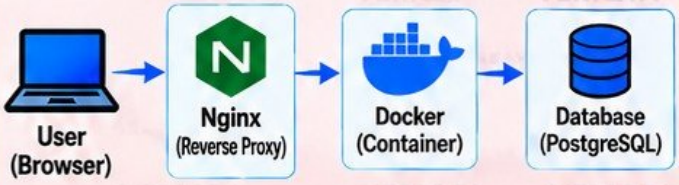
- Interviewers will ask you about your projects.
- They want to understand your thinking, problem-solving and technical depth.
- Clear explanations show real knowledge, not memorized answers.
- Helps you stand out from other candidates.
- Builds confidence in discussing your work.

Common Interview Questions

1. Why did you choose this architecture?
2. How did you test the deployment?
3. What failed, and how did you diagnose it?
4. What security improvements did you make?
5. What would you change for production?
6. Walk me through your project step by step.

Example Project Overview

Project: Containerized Web Application



How to Structure Your Answer

1. Start with a short project overview.
2. Explain the problem you were solving.
3. Talk about your architecture and why you chose it.
4. Describe how you built, tested and deployed it.
5. Discuss what failed and how you fixed it.
6. Explain security, monitoring and other improvements.
7. End with what you would change for production.

Sample Interview Questions and Answers

Q1: Why did you choose this architecture?

I chose a containerized architecture with Nginx, a Python application and PostgreSQL because it is simple, modular and easy to deploy and scale. It also reflects common real-world patterns.

Q2: How did you test the deployment?

I used automated tests in GitHub Actions, tested the application locally and in a staging environment, and verified that all services were running correctly using health checks and logs.

Q3: What failed, and how did you diagnose it?

The container initially exited due to a missing dependency. I checked the container logs, identified the error, updated the Dockerfile to install the dependency, rebuilt the image and verified the fix.

Q4: What security improvements did you make?

I used environment variables for secrets, ran the container as a non-root user, limited container permissions and added .gitignore to prevent sensitive files from being committed.

Q5: What would you change for production?

I would use a managed database, set up monitoring with Prometheus and Grafana, implement automated backups, use a CI/CD pipeline with deployment to Kubernetes and add proper alerting.

Tips for a Strong Interview Discussion

- Know your project end to end.
- Be honest about challenges and what you learned.
- Use real evidence (code, diagrams, logs, screenshots).
- Explain your thinking and trade-offs.
- Avoid memorized or generic answers.
- Be ready to discuss improvements.
- Keep your answers clear and concise.

Evidence You Can Show

- GitHub repository (code and README)
- Architecture diagram
- Screenshots of the running application
- Test results (CI pipeline logs)
- Logs or monitoring dashboard
- Documentation and setup instructions

Deliverable

A concise project walkthrough supported by real evidence:

- Short explanation of the project
- Key technical decisions
- How you tested and deployed it
- What failed and how you fixed it
- Improvements you would make

Key Takeaways

- Be prepared to explain your projects clearly.
- Use real examples, not memorized answers.
- Show your problem-solving process.
- Discuss challenges, fixes and improvements.
- Confidently connect your work to real engineering practices.

Career Value

- Helps you communicate technical work effectively.
- Demonstrates real engineering thinking.
- Shows you can discuss and defend your decisions.
- Increases your chances of passing technical interviews.
- Builds confidence for DevOps, SRE and platform engineering roles.

VERIQTA

The Junior DevOps Portfolio Readiness Test

Check your skills, find gaps and get ready for interviews

What This Test Is About

A practical self-assessment to help you know if your DevOps projects are complete, understandable and ready to discuss in interviews. Use this checklist to identify what you have done, what is missing and what needs verification.

How to Use This Test

1. Go through each item.
2. Mark it as Completed, Incomplete or Unverified.
3. Be honest about your current level.
4. Review the improvement suggestions for any gaps.
5. Complete all items before using your project in interviews.

Status Meaning

- **Completed** — You can do this confidently and have evidence.
- **Incomplete** — You have not done this yet or it needs work.
- **Unverified** — You think you can do this, but you have not recently tested or confirmed it.

Portfolio Readiness Checklist

#	Skill / Question	Completed	Incomplete	Unverified	Evidence to Check
1	Can you run the project from a clean environment?	☐	☐	☐	Follow your README on a new machine or cloud VM.
2	Can you explain its architecture?	☐	☐	☐	Architecture diagram and clear explanation of each component.
3	Can you demonstrate a passing test?	☐	☐	☐	Test output (unit, integration or pipeline tests) with success results.
4	Can you diagnose one realistic failure?	☐	☐	☐	Logs, error messages, root cause analysis and the fix you applied.
5	Can you explain the security boundaries?	☐	☐	☐	Description of network access, secrets handling, user permissions and other security controls.
6	Can you clean up the environment safely?	☐	☐	☐	Cleanup commands and proof that resources were removed (e.g. cloud resources, containers).

Example Evidence

- README (with setup steps)
- Architecture diagram
- Test output (success)
- Logs and screenshots
- Security configuration
- Cleanup confirmation

If You Marked Incomplete

- Review your project and identify what is missing.
- Complete the missing work.
- Push the changes to your repository.
- Test again and update your evidence.
- Revisit this checklist until all items are completed or verified.

If You Marked Unverified

- Run the project again from a clean environment.
- Re-check your test results.
- Verify that your explanations are accurate.
- Confirm that your security and cleanup steps work.
- Update your evidence and change the status to completed.

Key Takeaways

- A strong portfolio is complete, tested and well documented.
- You should be able to run, explain, troubleshoot and clean up your project.
- Real evidence builds credibility in interviews.
- This checklist helps you identify gaps and improve your skills.
- Complete all items to be ready for junior DevOps opportunities.

Career Value

- Demonstrates practical DevOps skills.
- Shows problem-solving and troubleshooting ability.
- Builds confidence for technical interviews.
- Helps you present your work clearly and professionally.
- Increases your chances of getting hired.

VERIQTA

Your 30-Day GitHub-to-DevOps Action Plan

A realistic four-week plan to build skills, complete a project and get ready for interviews

What You Will Achieve

1. Build real practical skills step by step.
2. Complete one strong, original portfolio project.
3. Be ready to discuss your work in interviews.
4. Have a clear path to two more projects.
5. Gain confidence as a junior DevOps engineer.

Tools You Will Use

GitHub · Linux · Docker · CI/CD · Kubernetes · Prometheus · Grafana · Documentation

Week 1

Days 1 – 7

GitHub, Linux Troubleshooting and Docker

- Set up and understand GitHub.
- Practice essential Linux troubleshooting.
- Learn and use Docker (Images, Containers, Volumes).
- Run a simple application in Docker.
- Push your code to GitHub with a clean README.

Outcome

Comfortable with GitHub, Linux and Docker basics.

Week 2

Days 8 – 14

CI Workflows and Application Testing

- Set up a CI workflow using GitHub Actions.
- Write and run automated tests.
- Build and push a Docker image in the pipeline.
- Fix a failing pipeline.
- Add test results to your project documentation.

Outcome

A working CI pipeline with automated tests for your application.

Week 3

Days 15 – 21

Kubernetes Fundamentals and Monitoring

- Learn core Kubernetes concepts (Pods, Deployments, Services, ConfigMaps).
- Deploy your application to a Kubernetes cluster (local or cloud).
- Set up monitoring with Prometheus and Grafana.
- Explore logs and basic troubleshooting.
- Simulate and resolve at least one failure.

Outcome

A running application on Kubernetes with basic monitoring.

Week 4

Days 22 – 30

Complete Project, Document and Prepare for Interviews

- Build an original end-to-end project (choose a simple real-world application).
- Document the architecture, setup steps, tests and failures.
- Improve security and configuration.
- Prepare a clear README with diagrams and screenshots.
- Practice explaining your project using the key interview questions.

Outcome

A strong, demonstrable portfolio project and interview readiness.

Example 30-Day Timeline

Days 1–7	GitHub, Linux, Docker (foundation)
Days 8–14	CI workflows and testing
Days 15–21	Kubernetes and monitoring
Days 22–30	Complete project, document, prepare for interviews

Final Outcome and Next Steps

1. One strong, original portfolio project (end-to-end).
2. A clear path to two additional projects (later, build on what you learned).
3. Real skills you can demonstrate in interviews.
4. A professional GitHub profile with well-documented work.
5. Increased confidence and better chances of getting hired as a junior DevOps engineer.

Important Guidance

- Focus on quality, not quantity.
- Build and document your own work.
- Learn from examples, but do not just copy repositories.
- Be honest about your experience.
- Show what you built, what you learned and what you can explain.
- Practice discussing your project clearly and confidently.

Key Takeaways

- A focused 30-day plan can take you from beginner to a strong junior DevOps portfolio.
- Complete one end-to-end project and prepare for interviews.
- Aim for two additional projects over time, rather than 10 copied repositories.
- Real skills, clear documentation and the ability to explain your work matter most.

Career Value

- Demonstrates practical, job-relevant skills.
- Shows real project experience from end to end.
- Builds confidence for technical interviews.
- Helps you stand out as a serious and prepared candidate.
- Provides a clear next step toward more advanced projects and roles.